ibs
INTELLIGENT BUILDING SOLUTIONS

eBilling MoMo Gateway

Staff User Manual

How to take MTN Mobile Money payments, follow up payments that are stuck, and produce reports from the gateway dashboard.

Dashboard address: `/dashboard/` on the gateway server

Version 1.2 · November 2026

All rights reserved. www.ibsgh.com

Contents

1. Signing in and out

2. Finding your way around

3. Dashboard

4. Taking a MoMo payment

5. Transactions

6. Checking payments with MTN

7. Reports

8. Callback Logs

9. Applications

10. What the statuses mean

11. Troubleshooting

12. For administrators

13. Adding an eBilling site

14. Updating the server

This manual is for IBS staff who use the eBilling MoMo Gateway dashboard to take MTN Mobile Money payments, follow them up and report on them.

The dashboard lives at **`/dashboard/`** on the gateway server (for example `https://www.ibsbill.org/dashboard/`).

1. Signing in and out
2. Finding your way around
3. Dashboard
4. Taking a MoMo payment
5. Transactions
6. Checking payments with MTN
7. Reports
8. Callback Logs
9. Applications
10. What the statuses mean
11. Troubleshooting
12. For administrators
13. Adding an eBilling site
14. Updating the server

01

Signing in and out

1. Open `/dashboard/` in your browser. You are taken to the sign-in page.
2. Enter your **username** and **password**, then select **Login**. Use the eye icon to show or hide the password while typing.
3. You land on the **Dashboard**.

Only **staff accounts** can sign in. If you see "*Your username and password didn't match, or the account isn't staff*", check your details, then ask an administrator to confirm your account has staff access (see For administrators).

To sign out, select the **log-out icon** next to your name at the top of the sidebar.

02

Finding your way around

Area	What it does
Sidebar (left)	Your name and the sign-out button at the top, then the pages: Dashboard, MoMo Payment, Transactions, Reports, Callback Logs, Applications and Django Admin. The page you are on is highlighted in green.
Toolbar (top)	The panel button hides or shows the sidebar. The breadcrumb (*Home › Transactions › …*) shows where you are; select any part of it to go back.
Page title	The large heading, with the page's main buttons on the right (for example **Export CSV**).

On a phone, the sidebar is hidden. Tap the panel button to open it and tap outside it to close it.

03

Dashboard

The Dashboard gives a quick picture of all payments.

- **Collected**: total of all successful payments, with the **deduction** taken from it (1%) and the **net** amount underneath.
- **Transactions**: number of payment requests, with how many are still pending.
- **Success rate**: successful payments as a share of payments that have finished (successful + failed). Pending payments are not counted.
- **Applications**: how many systems are registered to request payments.
- **Last 14 days**: bars show the number of requests per day; the line shows the amount collected.
- **By application**: amount collected and success count for each registered system.
- **Recent transactions**: the latest eight payments. Select a row to open it.
- **MTN callbacks** (at the top): when MTN last called back, how many callbacks arrived in the last 24 hours, and how many need attention (see Callback Logs). A yellow warning appears if payments have been pending for over 30 minutes with no callback at all, which usually means MTN cannot reach the gateway.

Buttons: **New MoMo payment** opens the payment page; **Export CSV** downloads every transaction.

04

Taking a MoMo payment

Use **MoMo Payment** in the sidebar to send a payment request to a customer's phone.

1. Choose the **Application** the payment belongs to (for example `diamond-ebilling`).
2. Enter the customer's **MoMo number**:

- 10 digits starting with `0` (for example `024 123 4567`), or
- 12 digits starting with `233` (for example `233241234567`).
- Spaces are fine. Numbers starting `020`, `027`, `050` or `057` are not MTN numbers and are refused.

3. Enter the **Amount** in cedis.
4. Enter the **Device ID**: the meter serial number being paid for.
5. **Username** is filled in with your username. Change it to the customer's account or room name if your team records it that way.
6. Select **Send payment request**.

The **Status** panel on the right then shows the progress:

Step	What you see
Request sent to MTN	MTN accepted the request. The customer's registered MoMo name and number are shown, so you can confirm you have the right person.
Customer approval	The customer gets a prompt on their phone and enters their MoMo PIN. The page checks with MTN every 10 seconds.
Final status	**Payment successful** or **Payment failed**. A failure shows the reason MTN gave, for example *the customer did not approve in time*, *not enough money in the wallet* or *the customer declined or entered the wrong PIN*.

If there is no answer after **5 minutes**, the page stops checking and shows **Still pending**. The payment is not lost: open it from **View transaction** later, or use Check status with MTN.

If you see **Request not sent**, MTN refused the request. The message says why; the most common causes are a wrong number or an MTN service problem. Check the number and try again. A request MTN refuses outright is recorded as **Failed**.

Sent it twice by accident? If the same application, device, number and amount is sent again within a minute while the first is still pending, the customer is **not** prompted a second time. You are shown the first payment instead.

05

Transactions

Transactions lists every payment request, newest first, **50 per page**. Use the page numbers at the bottom to move around: the current page is highlighted with the page before and after it, ‹ and › move one page, and « and » jump to the first and last page. Your filters are kept as you move between pages.

Finding a payment

- **Search** matches the reference, payer name, payer number, username, device ID or MTN transaction ID. Part of a value is enough (for example `kofi` or `0241`).

- **Status**: Successful, Pending or Failed.
- **App**: one registered application.
- **From / To**: request dates (both days included).
- Select **Filter** to apply the search and dates. Status and App apply as soon as you change them. **Clear** removes all filters.

The list

Column	Meaning
Date	When the request was made.
Reference	Our reference for the payment (hover to see it in full). Quote this to MTN or the eBilling team.
Payer	The payer's registered MoMo name, with their number underneath. *Name unavailable* means MTN did not return a name.
App, Username, Device	Which system, account and meter the payment was for.
Amount	The amount requested.
Status	Successful, Pending or Failed. Hover over **Failed** to see the reason.

Export CSV downloads the list with your current filters applied, including the **reason for failure**. The footer shows the total amount of the matching requests.

A single payment

Select a row to open it. The **Details** panel shows everything recorded, including the payer, the MTN transaction ID, the first response from MTN and, for a failed payment, the **reason for failure**. The **Lifecycle** panel shows each stage: request created, sent to MTN and final status. At the bottom, **Callbacks from MTN** lists every callback MTN sent about this payment and what the gateway made of it.

06

Checking payments with MTN

Payments normally update themselves when MTN reports the result. When a payment is stuck on **Pending**, you can ask MTN directly.

One payment

Open the payment and select **Check status with MTN** (top right; only shown while the payment is unfinished). A message tells you the result:

- *MTN reports SUCCESSFUL / FAILED. The transaction has been updated.*

- *MTN reports PENDING. Nothing changed.*: the customer has not answered yet.
- *MTN has no record of this reference*: the request never reached MTN.

Many payments at once

1. On **Transactions**, filter to the payments you want, for example **Status: Pending**, or a date range.
2. A yellow bar shows how many payments in the list are unfinished. Select **Check N with MTN** and confirm.
3. The payments are checked in small batches. A progress bar and counters show how many came back successful, failed, still pending, not found at MTN, or hit an error.
4. **Stop** halts after the current batch; everything already checked is kept.
5. When it finishes, select **Reload list**.

Mark ones MTN has no record of as failed: tick this only when cleaning up old requests. It marks requests that never reached MTN as **Failed** so they stop showing as pending. Leave it unticked for recent payments.

Checking also fills in the payer's number and MoMo name on older payments that don't have them yet, and records the reason for any payment MTN reports as failed.

07

Reports

Reports summarises payments for a period.

1. Pick the period type: **Monthly**, **Quarterly**, **Half-yearly**, **Yearly** or **Selected period**.
2. Pick the year and month, quarter (Q1 = Jan–Mar, Q2 = Apr–Jun, Q3 = Jul–Sep, Q4 = Oct–Dec) or half (H1 = Jan–Jun, H2 = Jul–Dec). For **Selected period**, pick the **From** and **To** dates.
3. Optionally pick one **Application**.
4. Select **Run report**.

The report shows:

- **Collected**, **Requests**, **Success rate** and **Average payment** (per successful payment). Under Collected you see the **deduction (1%)** and the **net** amount.
- A **▲ / ▼ percentage** comparing Collected and Requests with the previous period of the same length (for example Q2 against Q1, or March against February). A dash means the previous period had nothing to compare.
- A **trend chart**: by day for periods up to about two months, by month for longer ones.
- **Breakdown** table with a total row, showing for each day or month the requests, successful, failed and pending counts, **Rate (%)**, **Collected**, the **Deduction (1%)** and the **Net**. Select a day or month to see its transactions.
- **By application** table with the same columns.

The 1% deduction is worked out on **successful payments only** and rounded to 2 decimal places for each day or month, so the Total row is the sum of the rows above it. Failed and pending payments carry no deduction because no money was collected.

Export CSV downloads the breakdown, including the deduction and net columns.

Export PDF downloads a finished report to share or file: the IBS logo and watermark, the summary figures, a chart, the breakdown by day or month and the by-application table, stamped with the date and your username.

08

Callback Logs

When a customer approves, declines or ignores a payment prompt, MTN sends the gateway a **callback**. Every callback is recorded here exactly as received. Use it when investigating a disputed or stuck payment.

At the top of the page:

- **Last callback received** and **callbacks in the last 24 hours** show whether MTN is reaching the gateway. If the last callback was a long time ago while payments are being made, something is wrong. Tell an administrator.
- **Need attention (last 7 days)** counts callbacks that deserve a look. Select the number to see them.
- A yellow warning appears when payments from the last 3 days have been pending for over 30 minutes with **no callback at all**. MTN calls back for every request it accepts, so this points to the callback address not being reachable. In the meantime, use Check status with MTN to settle them.

The list

Each row shows when the callback arrived, **which payment it belongs to** (select the reference to open the payment, with its amount and status), what the callback **said**, the **outcome**, and the IP address it came from. It is shown 50 to a page, newest first.

- Search by text in the message (for example a number), by payment reference or by IP address.
- Filter by **outcome**, or choose **Needs attention** to see only the callbacks that deserve a look.

Outcomes

The gateway does not simply accept what a callback says. It uses the callback to find the payment, then **asks MTN** for the real result before recording it. The outcome says what happened:

Outcome	Meaning	What to do
Recorded	MTN confirmed the result and it was saved on the payment.	Nothing.

Outcome	Meaning	What to do
Already recorded	The payment already had its final result (MTN sometimes sends a callback twice).	Nothing.
Still pending	MTN says the customer has not answered yet.	Wait.
Not confirmed by MTN	The callback claimed a final result but MTN did not confirm it. Usually MTN's own status had not caught up yet.	Check the payment with MTN in a few minutes. If it keeps happening for one payment, treat it as suspicious.
Disagrees with MTN	The callback claimed one final result and MTN reports a different one. MTN's answer was recorded.	**Look into this.** Tell an administrator.
Unknown reference	The callback names a payment the gateway has no record of. MTN only calls about payments we sent.	Tell an administrator.
Could not check with MTN	The gateway could not reach MTN to confirm, so nothing was recorded.	Check the payment with MTN.
Not valid JSON	The message could not be read.	Tell an administrator.

Callbacks received before outcomes were recorded show a dash instead of an outcome.

A single callback

Select a row to see the full message (the payment reference, the status MTN reported, the payer and the transaction ID), its headers and where it came from. The page shows what the callback said against what MTN said when the gateway checked, links to the payment, and has a **Check status with MTN** button while the payment is unfinished.

A callback in this list is a record of what was received, not proof that a payment succeeded; always check the payment itself on **Transactions**.

09

Applications

Only **registered applications** can request payments through the gateway. Each has a name, a web address, the **IP addresses** it calls from and a key. When an application asks for a payment, the gateway checks that the **key** matches and that the request comes from one of the registered **IP addresses**; otherwise it is refused.

An application that runs on several servers needs all of their addresses. Enter them separated by commas, and use a range for a whole group, for example `10.0.4.170, 10.0.4.218, 10.0.5.0/24` (`/24` covers every address from `10.0.5.0` to `10.0.5.255`).

- **Register an application**: fill in the form on the right and select **Register**. Leave **Key** blank to generate a long random one. Copy the key with the clipboard icon and give it to the system's developers.
- **Edit**: change the details and select **Save changes**. If you change the key or the IP address, the application stops working until its developers update their side to match.
- **Remove app**: on the edit page. The application can no longer request payments; its past transactions are kept.

The table also shows how many requests each application has made and how much it has collected.

Where payment requests come from

eBilling asks the gateway for a payment **from the customer's own browser**, not from the eBilling server. Two things follow from that:

- The **Requesting IP** shown on a payment is the address of the customer's phone or computer. It changes from customer to customer and from network to network.
- For that reason the IP check is **switched off** for now (`ENFORCE_APP_IP=False`). Do not register one address and turn it on, or most customers would be refused. The check becomes useful only if eBilling's developers move the request to their server, where the address is fixed.

What protects the gateway meanwhile is the application **key**, the list of websites allowed to call it (see Adding an eBilling site) and the limit on requests per minute. Because the key is sent from the browser, use a long random one (leave **Key** blank when registering) and replace weak keys such as `diamond` when the site's developers can change their side at the same time.

10

What the statuses mean

Status	Meaning	What to do
Pending	Sent to MTN and waiting for the customer, or the result hasn't reached us yet.	Wait a few minutes, then check with MTN.
Successful	The customer approved and the money was collected.	Nothing on the gateway. The meter is credited on the eBilling side.
Failed	The payment did not go through. The reason is shown on the payment (see below).	Ask the customer to try again with a new request.

Reasons for a failed payment

MTN gives a reason with every failure. It is shown when you hover over **Failed** in the list, on the payment's page and on the payment screen.

Reason shown	What it means	What to do
Customer did not approve in time (PIN not entered before the prompt expired)	The prompt went to the phone but the customer did not enter their PIN in time.	Ask them to try again and watch for the prompt.
Customer declined the prompt, or entered the wrong PIN	They pressed cancel, or the PIN was wrong.	Ask them to try again, with the correct PIN.
Not enough money in the wallet	The MoMo balance is lower than the amount.	Ask them to top up, or pay a smaller amount.
Not a registered MTN MoMo account	The number has no MoMo wallet.	Check the number.
Customer's MoMo limit reached	A daily or monthly limit on their account.	Try again later or use another number.
MTN had a problem / MTN MoMo was unavailable	A fault on MTN's side.	Try again shortly.
A configuration problem (collection account, currency, callback address)	Something is wrong in the gateway's MTN settings.	Tell an administrator.

MTN does not always say exactly what went wrong. A customer who never enters their PIN usually shows as *did not approve in time*, and a wrong PIN usually shows as *declined or wrong PIN*. Older failed payments usually have no reason: MTN stops returning it after a while, and the backfill described in section 12 recovered it for only a handful. Failures from now on keep their reason.

11

Troubleshooting

Message or problem	Cause and fix
Could not get an access token from MTN	The gateway couldn't sign in to MTN. Usually the MTN credentials are wrong or MTN is down. Tell an administrator.
Enter a valid MTN MoMo number	The number isn't 10 digits starting with 0, or 12 digits starting with 233, or isn't an MTN number.
eBilling shows *MoMo payment not started: The MoMo gateway could not be reached*	The customer's browser was stopped before the request reached the gateway. Almost always the eBilling site's address is missing from `CORS_ALLOWED_ORIGINS` (see section 13). Press F12 in the browser, open **Console**, and retry: the red error names the address that was refused. If nothing is blocked, check that eBilling's `MOMO_REQUEST_URL` is `https://ibsbill.org/bill/`.
Choose a registered application	The application you picked isn't registered. Add it on **Applications**.

Message or problem	Cause and fix
An application is told *Invalid application credentials*	Its name, key or IP address doesn't match what is registered. Check the **Applications** page, or ask the application's developers to check what they send.
Too many requests	An application, or one IP address, sent far more requests than normal in a minute. Wait a minute and try again.
Payment stuck on **Pending**	The customer hasn't answered, or MTN's result didn't arrive. Use **Check status with MTN**.
Payer shows **Name unavailable**	MTN didn't return a name for that number. The payment is not affected.
Payer shows —	Older payment with no number recorded. Checking it with MTN fills it in.
A failed payment shows no reason	MTN didn't give one, or it is an older payment. Reasons for old payments usually cannot be recovered.
Signed out unexpectedly	Your session expired. Sign in again.

12

For administrators

Giving someone access

Staff accounts are managed in **Django Admin** (sidebar link, superusers only):

1. **Authentication and Authorization › Users › Add user**: set a username and password, then save.
2. On the next screen fill in their name and email, tick **Staff status**, then save.

To remove access, untick **Active** (keeps their history) rather than deleting the user.

The same Django Admin also lists **Applications**, **E bills** (the payments, searchable and filterable, with the status shown in colour) and **Call backs** (the callback logs, view only). Payments are best looked at on the dashboard; use the admin for corrections only.

Clearing pending payments automatically

The same check as the bulk button can be run on the server:

```
python manage.py recheck_pending --dry-run          # preview, saves nothing
python manage.py recheck_pending                    # record results
python manage.py recheck_pending --fail-not-found   # also fail requests MTN has no record of
```

It skips requests less than 10 minutes old (change with `--older-than MINUTES`). It is safe to schedule. On the live server this runs automatically every 15 minutes (`ibsbill-recheck.timer`). To install it on a new server, copy the two files in the `deploy/` folder to `/etc/systemd/system/`, make sure `User=`, `WorkingDirectory=` and the python path in `ibsbill-recheck.service` match the server, then run `sudo systemctl daemon-reload` and `sudo systemctl enable --now ibsbill-recheck.timer`.

Check it with:

```
systemctl list-timers | grep ibsbill           # shows the next run
sudo systemctl start ibsbill-recheck.service   # run it now
sudo journalctl -u ibsbill-recheck.service -n 20 --no-pager
```

A normal run says `No pending payments to check.`

To add the failure reason to older failed payments that don't have one:

```
python manage.py recheck_pending --backfill-reasons --dry-run   # preview
python manage.py recheck_pending --backfill-reasons
```

Run this once at most. MTN no longer gives a reason for older payments, so running it again finds nothing new.

The database

The live gateway keeps its data in **PostgreSQL**, on the same server: database `ibsbill`, login `ibsbill`. The connection settings (`DB_ENGINE`, `DB_NAME`, `DB_USER`, `DB_PASSWORD`, `DB_HOST`, `DB_PORT`) are in `/srv/ibsbill/.env`. Keep the password somewhere safe; it is not recorded anywhere else.

```
sudo systemctl status postgresql                      # is the database running?
cd /tmp && sudo -u postgres psql -d ibsbill -c '\dt'   # list the tables
```

If the gateway shows *Server error (500)* on every page and the log mentions `connection refused` or `password authentication failed`, check that PostgreSQL is running and that the password in `.env` is the one set for the `ibsbill` login.

The old SQLite file, `/srv/ibsbill/db.sqlite3`, was kept after the move as a fallback. It is no longer updated, so only use it to look something up from before the move. It can be deleted once nobody needs it.

Database backups

The gateway backs up its database every night at about 02:30 (`ibsbill-backup.timer`) and keeps the newest 14 backups in `/srv/ibsbill/backups`. Backup files are readable only by the gateway's user, because they contain every payment and the application keys.

To install the schedule on a server, copy `deploy/ibsbill-backup.service` and `deploy/ibsbill-backup.timer` to `/etc/systemd/system/`, set `User=` to the account the gateway runs as (find it with `systemctl cat ibsbill | grep ^User=`; on the live server it is `akpene`), and check that `WorkingDirectory=` and the python path match the server, then give that account the backups folder (replace `ACCOUNT`):

```
sudo mkdir -p /srv/ibsbill/backups
sudo chown -R ACCOUNT:ACCOUNT /srv/ibsbill/backups
sudo systemctl daemon-reload
sudo systemctl enable --now ibsbill-backup.timer
systemctl list-timers | grep ibsbill
```

To take a backup right now, for example before an update:

```
cd /srv/ibsbill && source venv/bin/activate
python manage.py backup_db
ls -lh backups/
```

A PostgreSQL database needs the `pg_dump` tool (`sudo apt install postgresql-client`). Backups are `ibsbill-<date>-<time>.dump` (PostgreSQL) or `.sqlite3.gz` (SQLite).

Copy the backups off the server. A backup on the same disk is lost if the server is. Copy the folder somewhere else regularly, for example to another machine or to cloud storage.

Restoring. Stop the gateway first (`sudo systemctl stop ibsbill`), restore, then start it again.

```
pg_restore --clean --if-exists --no-owner -d DBNAME backups/ibsbill-DATE-TIME.dump    #
PostgreSQL
gunzip -c backups/ibsbill-DATE-TIME.sqlite3.gz > db.sqlite3                           # SQLite
```

Practise a restore on a spare copy once, so it is not the first time during an emergency.

Settings an administrator may change

These go in the server's `.env` file (see README.md):

Setting	Effect
`COLLECTION_FEE_PERCENT`	The deduction shown in reports (default 1).
`CORS_ALLOWED_ORIGINS`	The eBilling websites allowed to ask for payments, separated by commas with no spaces, for example `https://diamondhostel.ebilling.ibsgh.com`. Each is `https://` plus the host name only: no path and no trailing `/`. **A site not listed here cannot take payments.**
`ENFORCE_APP_IP`	Whether a payment request must come from the application's registered IP address. **Keep it `False`** while eBilling calls from customers' browsers (see section 9).
`TRUSTED_PROXY_COUNT`	How many web servers sit in front of the gateway (default 1), so the real IP address is read correctly.
`MTN_CALLBACK_URL`	The public HTTPS address MTN calls with results: `https://ibsbill.org/bill/mtn-callback/`.

Setting	Effect
`CACHE_DIR`	Folder shared by the gateway's workers (default `./cache`, i.e. `/srv/ibsbill/cache`). It must be owned by the user the gateway runs as, with mode 700. A wrong owner makes every payment request fail with a server error.
`DB_ENGINE`	`sqlite` or `postgresql` (production).
`BACKUP_DIR`, `BACKUP_KEEP`	Where nightly database backups go (default `/srv/ibsbill/backups`) and how many of the newest to keep (default 14).

After changing `.env`, restart the gateway: `sudo systemctl restart ibsbill`.

13

Adding an eBilling site

A site can take payments only when **three things** agree: the gateway allows its address, it is registered as an application, and its own settings carry that application's name and key. Do them in this order, and test each site before starting the next.

Sites known so far: `diamondhostel` (live). Still to be added: `templesi`, `umat`, `mtnlo`, `signature`, `wesleytowers`, `alliedheights`, `capitalplace`.

1. Find the site's exact address. Open its eBilling in a browser and copy what is in the address bar, scheme and host only: `https://diamondhostel.ebilling.ibsgh.com`.

2. Allow it on the gateway. On the server, edit `/srv/ibsbill/.env` and add the address to `CORS_ALLOWED_ORIGINS`, keeping the existing ones, separated by commas with no spaces:

```
CORS_ALLOWED_ORIGINS=https://diamondhostel.ebilling.ibsgh.com,https://templesi.ebilling.ibsgh.com
```

Then `sudo systemctl restart ibsbill`, and check it:

```
curl -s -i -X OPTIONS https://ibsbill.org/bill/ \
  -H "Origin: https://templesi.ebilling.ibsgh.com" \
  -H "Access-Control-Request-Method: POST" \
  -H "Access-Control-Request-Headers: content-type" | grep -i access-control
```

You should see `access-control-allow-origin:` followed by the site's address. If nothing is printed, the address is not matched exactly.

3. Register it. In the dashboard open **Applications** and add the site. Use a name like `templesi-ebilling`, its web address, the site server's address in **IP addresses allowed** (for example `10.8.0.11`; it is not enforced now but must be valid), and leave **Key** blank. Copy the key that is generated.

4. Give the site its settings. In that site's eBilling configuration set the application name, the key, and `MOMO_REQUEST_URL=https://ibsbill.org/bill/` (HTTPS, with the trailing slash). Restart its eBilling if needed.

5. Test. Make a small real payment from the site, and check on **Transactions** that it appears under the right application and settles. If eBilling says *the MoMo gateway could not be reached*, see Troubleshooting.

Replacing a key

Change the key on **Applications** (edit, clear the key, save, copy the new one) and update the site's eBilling at the same moment. Between the two steps payments from that site are refused, so do it at a quiet time and make a test payment straight after.

14

Updating the server

Do these on the live server, in `/srv/ibsbill`, as the deployment user.

1. Back up the database first. Some updates change the database permanently.

```
cd /srv/ibsbill && source venv/bin/activate
python manage.py backup_db
```

The backup is written to `/srv/ibsbill/backups/`; check it with `ls -lh backups/`.

2. Get the new code and its requirements.

```
git pull origin main
source venv/bin/activate
pip install -r requirements.txt
```

3. Check the settings. Compare `.env` with the table in section 12 and add anything new. The cache folder must exist and belong to the gateway's user:

```
systemctl cat ibsbill | grep -E "^(User|Group)="
sudo mkdir -p /srv/ibsbill/cache
sudo chown <User>:<Group> /srv/ibsbill/cache
sudo chmod 700 /srv/ibsbill/cache
```

4. Update the database and static files, then restart.

```
python manage.py migrate
python manage.py collectstatic --noinput
python manage.py check
sudo systemctl restart ibsbill
sudo systemctl status ibsbill --no-pager
```

5. Check it works. Hard-refresh the dashboard (Ctrl+Shift+R) so the new stylesheet loads, then test the payment path without spending money:

```
curl -s -i -X POST https://ibsbill.org/bill/ -H "Content-Type: application/json" \
  -d '{"app":"diamond-ebilling","app_key":"
<key>","amount":0,"momo_number":"0241234567","device_id":"TEST"}'
```

`400 Invalid amount!` means the gateway is healthy. `403` means the name or key is wrong, `500` usually means the cache folder's owner is wrong, and `429` means too many requests.

When something is wrong

Look at	Command	What it tells you	
The gateway's log	`sudo journalctl -u ibsbill -f`	Every request and any error. Retry the payment while watching.	
nginx	`sudo tail -f /var/log/nginx/access.log`	grep /bill`	Whether payment requests arrive at all, and the answer given.
nginx errors	`sudo tail -n 30 /var/log/nginx/error.log`	Problems reaching the gateway or the eBilling servers.	
The timer	`systemctl list-timers`	grep ibsbill`	That stuck payments are being settled.

A payment attempt that shows only an `OPTIONS /bill/` line and no `POST /bill/` was blocked by the browser: the site is missing from `CORS_ALLOWED_ORIGINS`.

See README.md for installation, configuration and the API used by eBilling.